Lacuna: A BERT-Inspired Transformer Architecture for Missing Data Mechanism Classification

Progress Report — Architecture Specification

February 14, 2026

Abstract

We describe the architecture of Lacuna, a transformer-based classifier that determines the missing data mechanism (MCAR, MAR, or MNAR) operating on a given tabular dataset. Lacuna inherits core design principles from BERT [1]—bidirectional self-attention, pre-norm transformer layers, and masked prediction as a pretraining signal—but introduces fundamental modifications for the tabular missing-data domain: four-dimensional cell tokens, row-wise attention scope, hierarchical two-stage attention pooling, mechanism-specific reconstruction heads, explicit statistical missingness features, and a Mixture-of-Experts gating network that produces calibrated posterior probabilities over the three mechanism classes. The base configuration uses 4 transformer layers, 4 attention heads, hidden dimension 128, and achieves 82.6% classification accuracy with 901,130 parameters.

1 Introduction and Relationship to BERT

BERT (Bidirectional Encoder Representations from Transformers) [1] demonstrated that bidirectional self-attention over masked sequences produces powerful contextual representations for natural language understanding. Lacuna adapts this insight to a fundamentally different domain: tabular datasets with missing values. The core analogy is:

Concept	**BERT**	**Lacuna**
Input unit	Word token	Table cell (4D token)
Sequence	Sentence	Row of features
Masking	Random word masking	Natural + artificial cell masking
Prediction task	Predict masked word	Predict masked cell value
Classification task	Sentence-pair (NLI)	Mechanism class (MCAR/MAR/MNAR)
Sequence repr.	[CLS] token	Hierarchical attention pooling
Layers / Heads	12 / 12 (BERT-Base)	4 / 4 (Lacuna-Base)
Hidden dim	768	128
Parameters	110M	901K

The key insight enabling this transfer is that *MAR manifests as cross-column dependencies within rows*—precisely the kind of contextual relationship that self-attention excels at capturing. Under MAR, the probability of a value being missing in column j depends on observed values in other columns within the same row: $P(R_{ij} = 1 \mid X_{\text{obs}}) \neq P(R_{ij} = 1)$, where R_{ij} is the missingness indicator. Self-attention over features within a row can learn these dependencies, just as BERT's self-attention learns word-to-word dependencies within a sentence.

2 Input Representation

2.1 Token Structure

Unlike BERT, which maps discrete word tokens to dense vectors via vocabulary lookup, LACUNA operates on continuous-valued table cells. Each cell in the dataset is represented as a 4-dimensional token:

$$\mathbf{t}_{ij} = \begin{bmatrix} v_{ij} \\ o_{ij} \\ m_{ij} \\ p_j \end{bmatrix} \in \mathbb{R}^4 \tag{1}$$

where:

- $v_{ij} \in \mathbb{R}$: Robust-normalized value (approximately in $[-3, 3]$), set to 0 if missing.
- $o_{ij} \in \{0, 1\}$: Observation indicator (1 = observed, 0 = missing).
- $m_{ij} \in \{0, 1\}$: Mask type (0 = naturally missing, 1 = artificially masked for self-supervised pretraining).
- $p_j = j/(\text{max_cols} - 1) \in [0, 1]$: Normalized feature position.

The input to the model is a tensor of shape $[B, n, d, 4]$, where B is the batch size, $n \leq 128$ is the number of rows, and $d \leq 48$ is the number of features (columns). Both n and d are padded to maximum values with corresponding boolean masks $\mathbf{M}_{\text{row}} \in \{0, 1\}^{B \times n}$ and $\mathbf{M}_{\text{col}} \in \{0, 1\}^{B \times d}$.

Comparison to BERT. BERT's input is a sequence of discrete token IDs mapped to dense vectors through a learned vocabulary embedding. LACUNA instead receives continuous-valued cells with explicit binary indicators for observation status and mask type. This separation is critical: it enables the model to independently learn patterns like "high values go missing" (MNAR signal in v_{ij} conditioned on $o_{ij} = 0$) versus "missingness is uniform regardless of value" (MCAR signal).

3 Token Embedding Layer

The 4D token $\mathbf{t}_{ij}$ is projected to the hidden dimension $h = 128$ through four parallel embedding pathways, each producing a vector of dimension $h/4 = 32$:

$$\mathbf{e}_{ij}^{\text{val}} = \text{Linear}_{1 \to 32}(v_{ij} \cdot o_{ij}) \in \mathbb{R}^{32} \tag{2}$$

$$\mathbf{e}_{ij}^{\text{obs}} = \text{Embed}_{\{0,1\} \to 32}(o_{ij}) \in \mathbb{R}^{32} \tag{3}$$

$$\mathbf{e}_{ij}^{\text{mask}} = \text{Embed}_{\{0,1\} \to 32}(m_{ij}) \in \mathbb{R}^{32} \tag{4}$$

$$\mathbf{e}_{ij}^{\text{pos}} = \text{Embed}_{\{0,\ldots,31\} \to 32}(\lfloor p_j \cdot 31 \rfloor) \in \mathbb{R}^{32} \tag{5}$$

Note that in Equation (2), the raw value v_{ij} is scaled by the observation indicator o_{ij}, so missing cells contribute zero to the value embedding. This prevents the model from leaking information about placeholder values.

The four components are concatenated and projected:

$$\mathbf{h}_{ij}^{(0)} = \text{Dropout}_{0.1}\Big(\text{LayerNorm}\Big(\text{Linear}_{128 \to 128}\Big([\mathbf{e}_{ij}^{\text{val}}; \mathbf{e}_{ij}^{\text{obs}}; \mathbf{e}_{ij}^{\text{mask}}; \mathbf{e}_{ij}^{\text{pos}}]\Big)\Big)\Big) \in \mathbb{R}^{128} \tag{6}$$

Comparison to BERT. BERT sums three embeddings (token + position + segment). LACUNA concatenates four embeddings and projects, because the four token components carry orthogonal information that should not be mixed before the projection. BERT uses segment embeddings for sentence-pair tasks; LACUNA replaces these with mask-type embeddings that distinguish natural missingness from artificial masking.

4 Transformer Encoder

4.1 Architecture Overview

The encoder consists of $L = 4$ stacked transformer layers, each applying pre-norm multi-head self-attention followed by a position-wise feedforward network. The output shape is preserved: $[B, n, d, h]$ at every layer.

Table 1: Encoder hyperparameters for the three model variants.

Parameter	**Mini**	**Base**	**Large**
Hidden dimension (h)	64	**128**	256
Evidence dimension (p)	32	**64**	128
Transformer layers (L)	2	**4**	6
Attention heads (H)	2	**4**	8
Head dimension ($d_k = h/H$)	32	**32**	32
FFN inner dimension (d_{ff})	256	**512**	1024
Max columns ($d_{\max}$)	32	**32**	64
Dropout	0.1	**0.1**	0.1
Row pooling	mean	**attention**	attention
Dataset pooling	mean	**attention**	attention

4.2 Row-Wise Multi-Head Self-Attention

A critical departure from BERT: LACUNA applies self-attention *within each row independently*, not across the entire input. The input tensor is reshaped from $[B, n, d, h]$ to $[Bn, d, h]$, treating each row as a separate sequence of d feature tokens.

For layer $\ell \in \{1, \ldots, L\}$, the pre-norm attention computation is:

$$\hat{\mathbf{H}}^{(\ell)} = \text{LayerNorm}\Big(\mathbf{H}^{(\ell-1)}\Big) \tag{7}$$

$$\mathbf{Q} = \hat{\mathbf{H}}^{(\ell)}\mathbf{W}_Q^{(\ell)}, \quad \mathbf{K} = \hat{\mathbf{H}}^{(\ell)}\mathbf{W}_K^{(\ell)}, \quad \mathbf{V} = \hat{\mathbf{H}}^{(\ell)}\mathbf{W}_V^{(\ell)} \tag{8}$$

where $\mathbf{W}_Q^{(\ell)}, \mathbf{W}_K^{(\ell)}, \mathbf{W}_V^{(\ell)} \in \mathbb{R}^{128\times 128}$ are learned projection matrices. Each is conceptually split into $H = 4$ heads of dimension $d_k = 32$:

$$\mathbf{Q} = [\mathbf{Q}_1; \ldots; \mathbf{Q}_4], \quad \mathbf{K} = [\mathbf{K}_1; \ldots; \mathbf{K}_4], \quad \mathbf{V} = [\mathbf{V}_1; \ldots; \mathbf{V}_4] \tag{9}$$

where $\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i \in \mathbb{R}^{d\times 32}$ for each head i.

The scaled dot-product attention for each head is:

$$\text{head}_i = \text{softmax}\left(\frac{\mathbf{Q}_i\mathbf{K}_i^\top}{\sqrt{d_k}} + \mathbf{M}_{\text{attn}}\right)\mathbf{V}_i \in \mathbb{R}^{d\times 32} \tag{10}$$

where $d_k = 32$ is the head dimension and $\mathbf{M}_{\text{attn}} \in \mathbb{R}^{d\times d}$ is an attention mask that sets padding positions to $-\infty$ before the softmax. The attention weight matrix $\mathbf{A}_i = \text{softmax}(\mathbf{Q}_i\mathbf{K}_i^\top/\sqrt{d_k} + \mathbf{M}_{\text{attn}}) \in \mathbb{R}^{d\times d}$ has entry (j, k) representing how much feature j attends to feature k within the same row.

The heads are concatenated and projected:

$$\text{MHA}(\hat{\mathbf{H}}^{(\ell)}) = [\text{head}_1; \ldots; \text{head}_4]\,\mathbf{W}_O^{(\ell)} \in \mathbb{R}^{d\times 128} \tag{11}$$

where $\mathbf{W}_O^{(\ell)} \in \mathbb{R}^{128\times 128}$ is the output projection. The residual connection then yields:

$$\tilde{\mathbf{H}}^{(\ell)} = \mathbf{H}^{(\ell-1)} + \text{Dropout}_{0.1}\Big(\text{MHA}(\hat{\mathbf{H}}^{(\ell)})\Big) \tag{12}$$

Why row-wise, not full-dataset attention? Under MAR, missingness in column j depends on values in *other columns within the same row*: $P(R_{ij} \mid X_{i,\text{obs}})$. Cross-row attention would capture dataset-level summary statistics (e.g., "column 1 is 30% missing") but not the row-level dependencies that define MAR. Additionally, row-wise attention has $O(d^2)$ memory per row, versus $O(n^2d^2)$ for full-dataset attention, enabling processing of datasets with hundreds of rows.

4.3 Position-Wise Feedforward Network

After attention, each token passes through a two-layer feedforward network with GELU activation:

$$\hat{\tilde{\mathbf{H}}}^{(\ell)} = \text{LayerNorm}\Big(\tilde{\mathbf{H}}^{(\ell)}\Big) \tag{13}$$

$$\text{FFN}(\hat{\tilde{\mathbf{H}}}^{(\ell)}) = \text{Linear}_{512\to 128}\Big(\text{Dropout}_{0.1}\Big(\text{GELU}\Big(\text{Linear}_{128\to 512}\Big(\hat{\tilde{\mathbf{H}}}^{(\ell)}\Big)\Big)\Big)\Big) \tag{14}$$

$$\mathbf{H}^{(\ell)} = \tilde{\mathbf{H}}^{(\ell)} + \text{Dropout}_{0.1}\Big(\text{FFN}(\hat{\tilde{\mathbf{H}}}^{(\ell)})\Big) \in \mathbb{R}^{d\times 128} \tag{15}$$

The FFN expands the hidden dimension by a factor of 4 ($128 \to 512$) before projecting back, following standard transformer practice.

Comparison to BERT. BERT-Base uses the same FFN structure but with dimensions 768 $\to$ 3072 $\to$ 768. Both use pre-norm residual connections, though original BERT used post-norm; LACUNA adopts pre-norm for improved training stability with fewer layers. BERT uses GELU activation; LACUNA does the same.

5 Hierarchical Attention Pooling

After $L = 4$ transformer layers, the output has shape $[B, n, d, 128]$—a representation for every cell in every row. This must be compressed to a single *evidence vector* $\mathbf{z} \in \mathbb{R}^{64}$ summarizing the entire dataset. LACUNA accomplishes this through two-stage hierarchical attention pooling, replacing BERT's [CLS] token approach.

5.1 Stage 1: Row Pooling (Features → Rows)

For each row i, the d feature representations $\{\mathbf{h}_{i1}^{(L)}, \ldots, \mathbf{h}_{id}^{(L)}\}$ are aggregated into a single row vector via learned attention:

$$\alpha_{ij} = \frac{\exp\left(s(\mathbf{h}_{ij}^{(L)})\right)}{\sum_{k=1}^{d} \exp\left(s(\mathbf{h}_{ik}^{(L)})\right) \cdot [\mathbf{M}_{\text{col}}]_k} \tag{16}$$

$$\mathbf{r}_i = \sum_{j=1}^{d} \alpha_{ij} \cdot \mathbf{h}_{ij}^{(L)} \in \mathbb{R}^{128} \tag{17}$$

where $s : \mathbb{R}^{128} \to \mathbb{R}$ is a learned scoring function:

$$s(\mathbf{x}) = \text{Linear}_{64\to1}(\tanh(\text{Linear}_{128\to64}(\mathbf{x}))) \tag{18}$$

This transforms the tensor from $[B, n, d, 128]$ to $[B, n, 128]$.

5.2 Stage 2: Dataset Pooling (Rows → Dataset)

The n row representations $\{\mathbf{r}_1, \ldots, \mathbf{r}_n\}$ are aggregated identically into a single dataset vector:

$$\beta_i = \frac{\exp(s'(\mathbf{r}_i))}{\sum_{k=1}^{n} \exp(s'(\mathbf{r}_k)) \cdot [\mathbf{M}_{\text{row}}]_k} \tag{19}$$

$$\mathbf{z} = \text{LayerNorm}\left(\text{Linear}_{128\to64}\left(\text{LayerNorm}\left(\sum_{i=1}^{n} \beta_i \cdot \mathbf{r}_i\right)\right)\right) \in \mathbb{R}^{64} \tag{20}$$

where s' has the same architecture as s (Equation 18) but with separate parameters.

Why not a [CLS] token? Tabular data has no privileged "start of sequence" position. A [CLS] token prepended to each row would need to learn to attend to the most informative features without positional bias. The learned attention pooling explicitly computes importance weights, allowing the model to learn that, e.g., "rows with partial missingness are most informative for MAR detection" and "features with extreme values matter most for MNAR detection."

6 Reconstruction Heads

Before classification, LACUNA employs a set of $K = 3$ reconstruction heads (in the default 1/1/1 symmetric configuration) that attempt to predict the values of missing cells. These serve a dual purpose: (1) as a self-supervised pretraining signal and (2) as a *discrimination signal*—the pattern of reconstruction errors across the different heads reveals the underlying mechanism.

Each head receives the token-level representations $\mathbf{H}^{(L)} \in \mathbb{R}^{B\times n\times d\times 128}$ and produces per-cell value predictions. The heads differ in their architectural inductive biases:

6.1 MCAR Head

A simple MLP applied independently to each token:

$$\hat{v}_{ij}^{\text{MCAR}} = \text{Linear}_{64\to1}\Big(\text{GELU}\Big(\text{LayerNorm}\Big(\text{Linear}_{128\to64}\Big(\mathbf{h}_{ij}^{(L)}\Big)\Big)\Big)\Big) \tag{21}$$

This head exploits no cross-column structure and can only use global distributional information encoded in each token's representation.

6.2 MAR Head (Cross-Attention to Raw Observed Values)

The MAR head uses a cross-attention mechanism where queries and keys are derived from the learned representations, but the *values* are the raw observed data:

$$\mathbf{Q}^{\text{mar}} = \mathbf{H}^{(L)}\mathbf{W}_Q^{\text{mar}} \in \mathbb{R}^{d\times64} \tag{22}$$

$$\mathbf{K}^{\text{mar}} = \mathbf{H}^{(L)}\mathbf{W}_K^{\text{mar}} \in \mathbb{R}^{d\times64} \tag{23}$$

$$\mathbf{V}^{\text{mar}} = \text{Linear}_{1\to64}(\mathbf{X}_{\text{raw}}) \in \mathbb{R}^{d\times64} \tag{24}$$

where $\mathbf{X}_{\text{raw}} \in \mathbb{R}^{d\times1}$ contains the raw observed values (v_{ij} from the input tokens). Attention is masked to attend *only to observed cells*:

$$\mathbf{C}_{ij} = \text{softmax}\left(\frac{\mathbf{Q}_j^{\text{mar}}(\mathbf{K}^{\text{mar}})^\top}{\sqrt{64}} + \mathbf{M}_{\text{obs}}\right)\mathbf{V}^{\text{mar}} \in \mathbb{R}^{64} \tag{25}$$

where $[\mathbf{M}_{\text{obs}}]_{jk} = -\infty$ if cell k is unobserved, and 0 otherwise. The context $\mathbf{C}_{ij}$ is then concatenated with the original representation and passed through a predictor MLP:

$$\hat{v}_{ij}^{\text{MAR}} = \text{MLP}_{192\to64\to1}\Big([\mathbf{C}_{ij};\mathbf{h}_{ij}^{(L)}]\Big) \tag{26}$$

Under MAR, this head should achieve *lower* reconstruction error than the MCAR head on naturally missing cells, because the cross-attention can learn to use observed values in correlated columns to predict missing ones. Under MNAR, both heads perform similarly because the missing value depends on itself, not on other columns.

6.3 MNAR Self-Censoring Head

This head models the hypothesis that extreme values are systematically censored:

$$\hat{v}_{ij}^{\text{base}} = \text{MLP}_{128\to64\to1}\Big(\mathbf{h}_{ij}^{(L)}\Big) \tag{27}$$

$$c_{ij} = \gamma\cdot\tanh\Big(\text{MLP}_{128\to64\to1}\Big(\mathbf{h}_{ij}^{(L)}\Big)\Big) \tag{28}$$

$$\hat{v}_{ij}^{\text{MNAR}} = \hat{v}_{ij}^{\text{base}} + c_{ij}\cdot(1-o_{ij}) \tag{29}$$

where γ is a learnable censoring scale parameter. The censoring adjustment c_{ij} is only applied to missing cells ($o_{ij} = 0$), learning whether missing values tend to be higher or lower than the population mean.

6.4 Discrimination Signal

The key insight: reconstruction errors on *naturally missing* cells (not artificially masked) differ across heads depending on the true mechanism:

True Mechanism	ϵ_{MCAR}	ϵ_{MAR}	ϵ_{MNAR}
MCAR	Moderate	Moderate	Moderate
MAR	Higher	**Lower**	Higher
MNAR	Variable	Higher	**Lower**

The error vector $\boldsymbol{\epsilon} = [\epsilon_{\text{MCAR}}, \epsilon_{\text{MAR}}, \epsilon_{\text{MNAR}}] \in \mathbb{R}^3$ is passed to the gating network as a discrimination signal.

7 Missingness Feature Extractor

While reconstruction errors discriminate MAR from MNAR, they are insufficient for MCAR vs. MAR discrimination (the cross-attention improves prediction under both, with a small gap). To address this, LACUNA computes 16 explicit statistical features from the raw data, organized into five groups:

Table 2: Missingness pattern features (16 total, non-learnable).

Group	**Features**	**Count**	**Primary Signal**
Missing rate statistics	mean, var, range, max of per-col rates	4	MCAR vs MAR
Point-biserial corr.	mean, max, std of $\text{corr}(R_j, X_k)$	3	MCAR vs MAR
Cross-col miss. corr.	mean, max cross-corr, frac. high	3	MCAR vs MAR
Distributional stats	mean skew, mean kurt, var of each	4	MAR vs MNAR
Little's MCAR approx.	chi-sq statistic, significance proxy	2	MCAR vs non-MCAR

The missing rate variance alone achieves Cohen's $d > 9.0$ between MCAR and MAR (MCAR produces nearly uniform per-column missing rates; MAR produces highly variable rates). These features require no learned parameters.

The feature vector $\mathbf{f} \in \mathbb{R}^{16}$ is concatenated with the evidence vector and reconstruction errors as input to the gating network.

8 Mixture of Experts and Classification

8.1 Gating Network

The Mixture of Experts (MoE) layer receives three concatenated input signals:

$$\mathbf{g}_{\text{input}} = [\mathbf{z}; \boldsymbol{\epsilon}; \mathbf{f}] \in \mathbb{R}^{83} \tag{30}$$

where $\mathbf{z} \in \mathbb{R}^{64}$ is the evidence vector (Section 5), $\boldsymbol{\epsilon} \in \mathbb{R}^3$ is the reconstruction error vector (Section 6), and $\mathbf{f} \in \mathbb{R}^{16}$ is the missingness feature vector (Section 7).

The gating network is a 2-layer MLP producing logits over $K = 3$ experts:

$$\mathbf{u} = \text{Dropout}_{0.1}(\text{GELU}(\text{LayerNorm}(\text{Linear}_{83\to 64}(\mathbf{g}_{\text{input}})))) \in \mathbb{R}^{64} \tag{31}$$

$$\boldsymbol{\ell} = \text{Linear}_{64\to 3}(\mathbf{u}) \in \mathbb{R}^3 \tag{32}$$

8.2 Temperature-Scaled Softmax

Expert probabilities are obtained via temperature-scaled softmax:

$$\pi_k = \frac{\exp(\ell_k/T)}{\sum_{k'=1}^{3} \exp(\ell_{k'}/T)}, \quad k \in \{\text{MCAR}, \text{MAR}, \text{MNAR}\} \tag{33}$$

The temperature T is set post-hoc to $T = 1.96$ based on calibration on a held-out validation set. This calibration unexpectedly improved both expected calibration error (ECE = 0.038) *and* classification accuracy, by softening overconfident wrong predictions.

8.3 Expert-to-Class Mapping

In the default 1/1/1 symmetric configuration (one expert per mechanism class), the expert probabilities map directly to class posteriors:

$$P(\text{class} = c \mid \mathcal{D}) = \pi_c, \quad c \in \{0, 1, 2\} \tag{34}$$

With the balanced "mean" aggregation, the posteriors are normalized by the number of experts per class to prevent structural bias (this matters when using > 1 MNAR variant).

8.4 Bayes-Optimal Decision Rule

Given the posterior $\mathbf{p} = [P(\text{MCAR}), P(\text{MAR}), P(\text{MNAR})]$, the model selects an action $a^* \in \{\text{Green}, \text{Yellow}, \text{Red}\}$ that minimizes expected loss:

$$a^* = \arg\min_a \sum_{c\in\{0,1,2\}} P(c \mid \mathcal{D}) \cdot L[a, c] \tag{35}$$

The loss matrix $\mathbf{L} \in \mathbb{R}^{3\times 3}$ encodes the cost of choosing action a when the true mechanism is c:

$$\mathbf{L} = \begin{bmatrix} 0.0 & 0.3 & 1.0 \\ 0.2 & 0.0 & 0.2 \\ 1.0 & 0.3 & 0.0 \end{bmatrix} \quad \begin{array}{l} \text{Row 0: Green (assume MCAR)} \\ \text{Row 1: Yellow (assume MAR)} \\ \text{Row 2: Red (assume MNAR)} \end{array} \tag{36}$$

The matrix is asymmetric: treating MNAR data as MCAR (top-right, $L = 1.0$) is the costliest error, while the Yellow action (MAR assumption) is the safest default with moderate cost under both MCAR and MNAR.

9 Training

9.1 Multi-Task Loss

The total training loss combines classification and reconstruction objectives:

$$\mathcal{L} = \lambda_{\text{cls}} \cdot \mathcal{L}_{\text{cls}} + \lambda_{\text{recon}} \cdot \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{aux}} \tag{37}$$

where:

- $\mathcal{L}_{\text{cls}}$: Cross-entropy loss on mechanism class posterior vs. ground-truth label ($\lambda_{\text{cls}} = 1.0$).
- $\mathcal{L}_{\text{recon}}$: MSE on artificially-masked cell predictions, averaged over all heads ($\lambda_{\text{recon}} = 0.5$).
- $\mathcal{L}_{\text{aux}}$: Auxiliary losses (load balancing, entropy regularization, KL divergence for the MNAR latent head).

9.2 Training Configuration

Table 3: Production training configuration.

Parameter	**Value**
Epochs	100
Batch size	16
Batches per epoch	200
Learning rate	3×10^{-4}
Optimizer	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Weight decay	0.01
Gradient clipping	1.0
Warmup steps	400
Early stopping patience	20 epochs
Training datasets	24 (UCI/OpenML)
Validation datasets	7
Missingness generators	110 total (32 MCAR, ∼36 MAR, ∼42 MNAR)
Total parameters	901,130
Post-hoc temperature	$T = 1.96$

10 Complete Forward Pass

Algorithm 1 summarizes the complete forward pass with tensor shapes.

11 Summary of Architectural Dimensions

Table 4 provides a complete reference of tensor shapes through the network.

12 Figures

References

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *NAACL-HLT*, 2019.

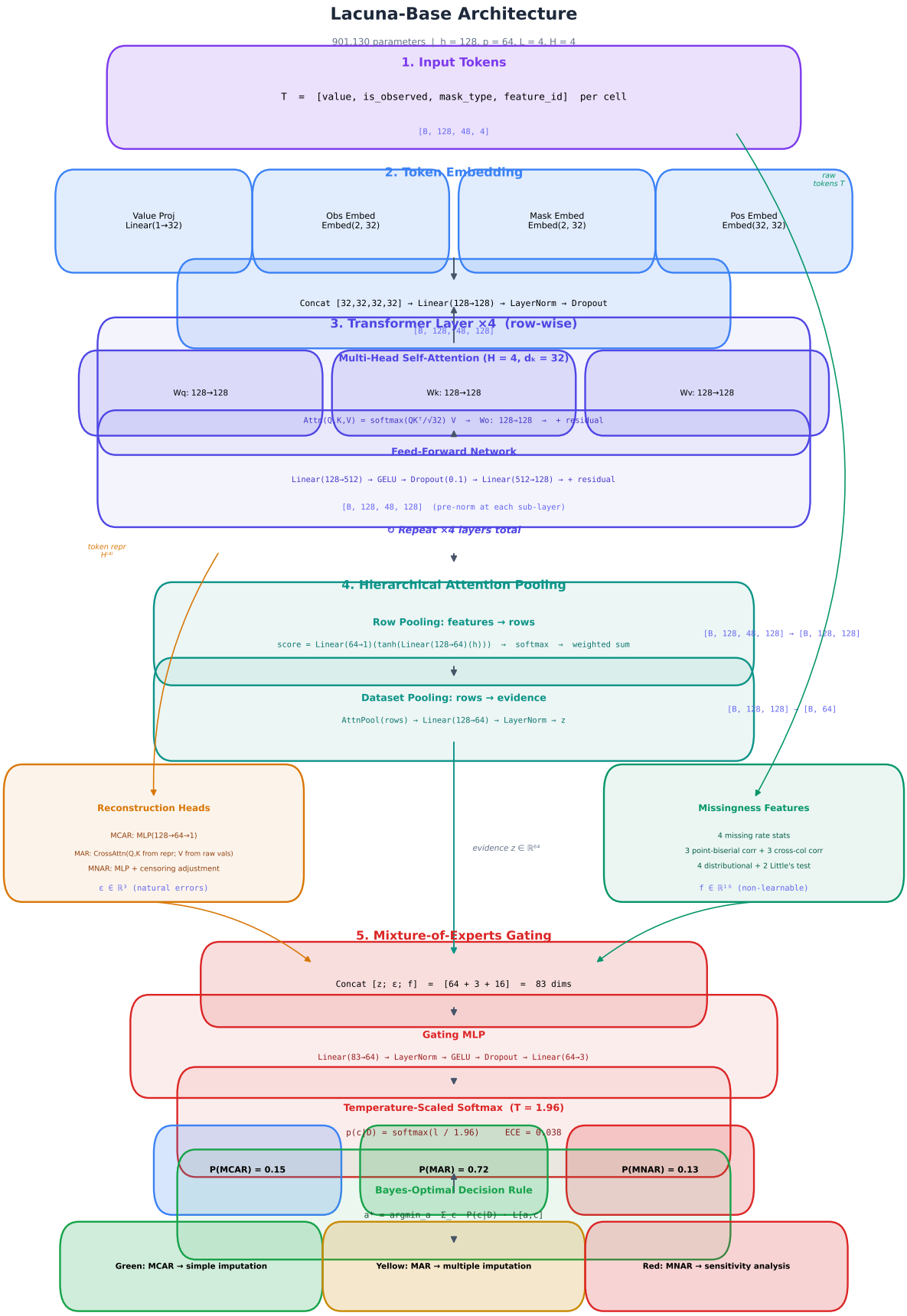

Figure 1: Complete LACUNA-Base architecture diagram showing the five-stage pipeline: (1) token embedding, (2) transformer encoder with row-wise attention, (3) hierarchical pooling, (4) recon-

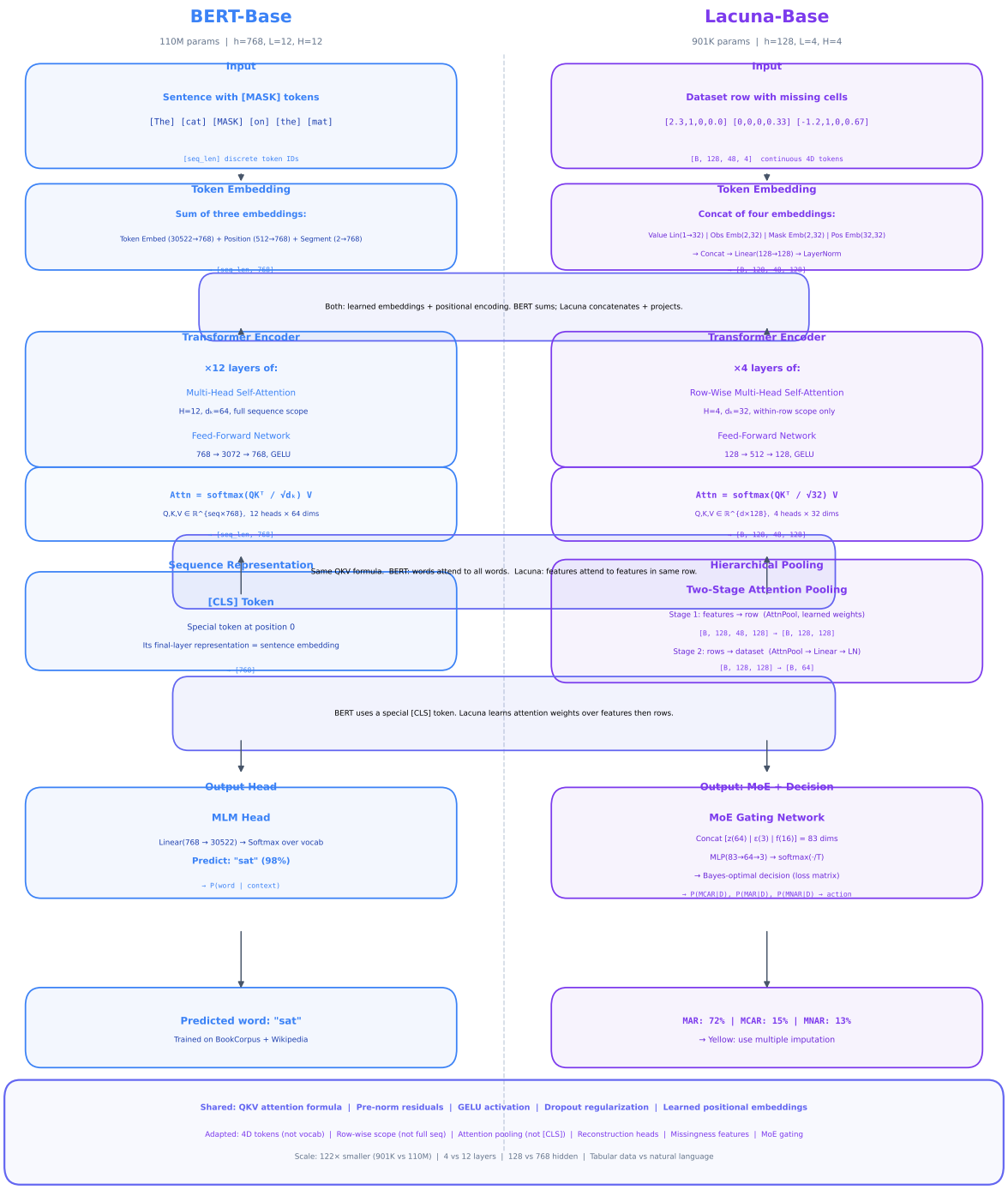

Figure 2: Side-by-side architectural comparison of BERT-Base and LACUNA-Base. Shared design elements (blue shading) include bidirectional self-attention, pre-norm transformer layers, and residual connections. Key adaptations for the tabular domain (purple shading) include 4D cell tokens, row-wise attention scope, hierarchical attention pooling, and mechanism-specific reconstruction heads. LACUNA is approximately 122× smaller than BERT-Base (901K vs. 110M parameters) while sharing the same fundamental transformer architecture.

Algorithm 1 LACUNA Forward Pass (Base Configuration)

Require: Token batch $\mathbf{T} \in \mathbb{R}^{B\times 128\times 48\times 4}$, masks $\mathbf{M}_{\text{row}}, \mathbf{M}_{\text{col}}$
Ensure: Posterior $\mathbf{p} \in [0,1]^{B\times 3}$, decision $a \in \{0,1,2\}^B$

Token Embedding: $\mathbf{H}^{(0)} \leftarrow \text{TokenEmbed}(\mathbf{T}) \in \mathbb{R}^{B\times 128\times 48\times 128}$
for $\ell = 1$ to 4 **do**
  Self-Attention: $\tilde{\mathbf{H}}^{(\ell)} \leftarrow \mathbf{H}^{(\ell-1)} + \text{MHA}(\text{LayerNorm}(\mathbf{H}^{(\ell-1)}))$
  FFN: $\mathbf{H}^{(\ell)} \leftarrow \tilde{\mathbf{H}}^{(\ell)} + \text{FFN}(\text{LayerNorm}(\tilde{\mathbf{H}}^{(\ell)}))$
end for
Row Pooling: $\mathbf{R} \leftarrow \text{AttnPool}_{\text{col}}(\mathbf{H}^{(4)}) \in \mathbb{R}^{B\times 128\times 128}$
Dataset Pooling: $\mathbf{z} \leftarrow \text{LayerNorm}(\text{Linear}_{128\to 64}(\text{AttnPool}_{\text{row}}(\mathbf{R}))) \in \mathbb{R}^{B\times 64}$
Reconstruction: $\boldsymbol{\epsilon} \leftarrow \text{ReconHeads}(\mathbf{H}^{(4)}, \mathbf{T}) \in \mathbb{R}^{B\times 3}$
Miss. Features: $\mathbf{f} \leftarrow \text{StatFeatures}(\mathbf{T}) \in \mathbb{R}^{B\times 16}$
Gating: $\boldsymbol{\ell} \leftarrow \text{GateMLP}([\mathbf{z}; \boldsymbol{\epsilon}; \mathbf{f}]) \in \mathbb{R}^{B\times 3}$
Posterior: $\mathbf{p} \leftarrow \text{softmax}(\boldsymbol{\ell}/T) \in [0,1]^{B\times 3}, \quad T = 1.96$
Decision: $a \leftarrow \arg\min_a \mathbf{p} \cdot \mathbf{L}[a, :]$
return $\mathbf{p}, a$

Table 4: Tensor shapes through the LACUNA-Base forward pass.

Stage	**Output Shape**	**Notes**
Raw input tokens	$[B, 128, 48, 4]$	4D tokens per cell
Token embedding	$[B, 128, 48, 128]$	Concat 4×32, project
Each QKV projection	$[B{\cdot}128, 4, 48, 32]$	Per-head, row-wise
Attention weights	$[B{\cdot}128, 4, 48, 48]$	$d \times d$ per head per row
Transformer output	$[B, 128, 48, 128]$	After $L = 4$ layers
Row pooling	$[B, 128, 128]$	Attention over columns
Dataset pooling	$[B, 64]$	Attention over rows + projection
Reconstruction errors	$[B, 3]$	One scalar per head
Missingness features	$[B, 16]$	5 groups, 16 features total
Gate input	$[B, 83]$	$64 + 3 + 16$
Gate hidden	$[B, 64]$	After first MLP layer
Class logits	$[B, 3]$	Before temperature scaling
Class posterior	$[B, 3]$	After softmax($\cdot$/1.96)
Expected risks	$[B, 3]$	Per action
Decision	$[B]$	$\{0, 1, 2\}$